

VLLM compile Deep Dive (it's all torch.compile)

By: **Ayush Satyam**
Associate MLE at  Red Hat

vLLM Compile

Always has been

it's all torch.compile



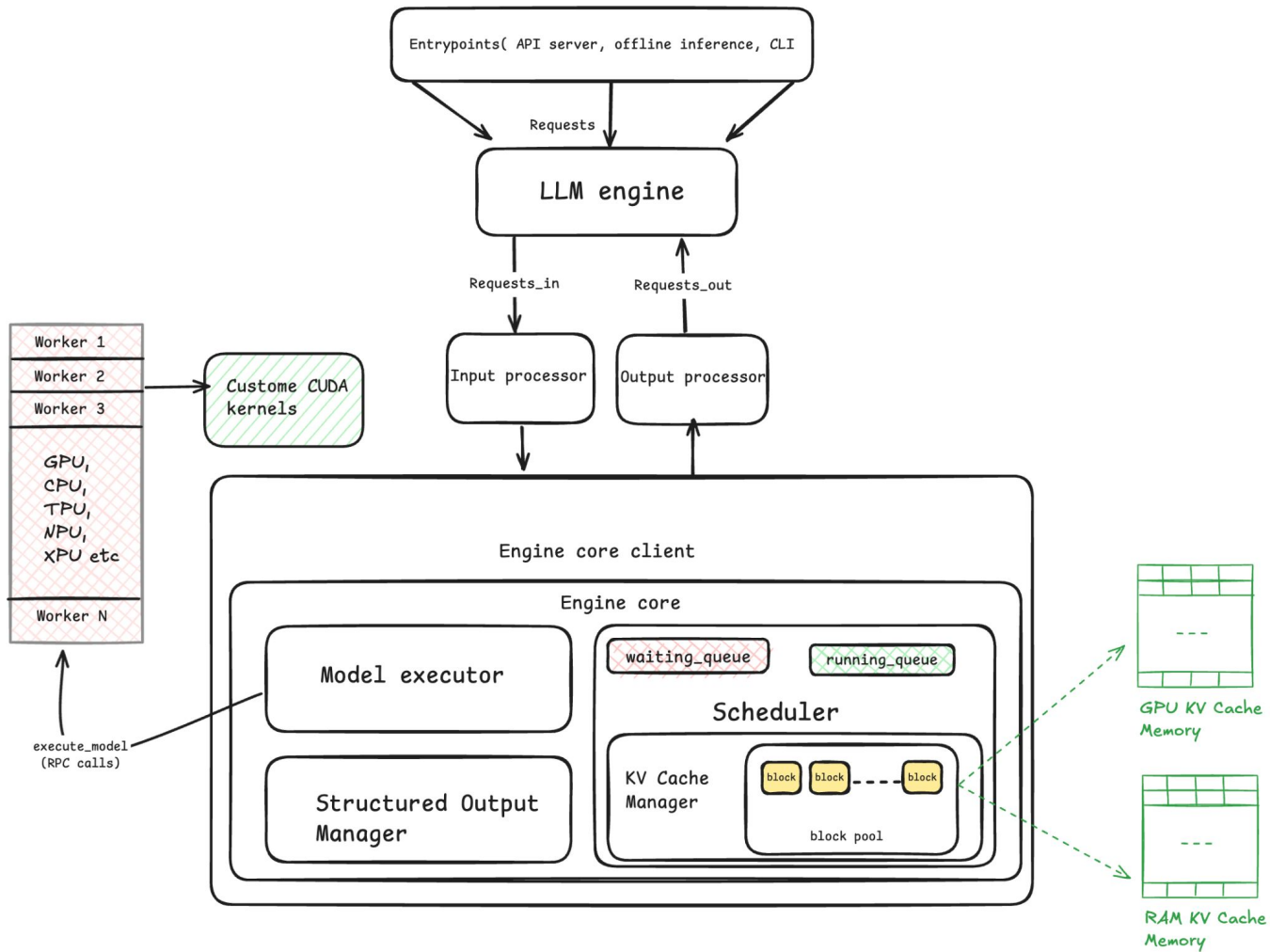
what is vLLM?

vLLM is a fast and easy-to-use library for LLM inference and serving.

Key innovations in vLLM?

- Paged Attention
- Optimized K-V Caching
- Optimized CUDA kernels
- Speculative Decoding
- Chunked Prefill and many more

System Architecture of vLLM



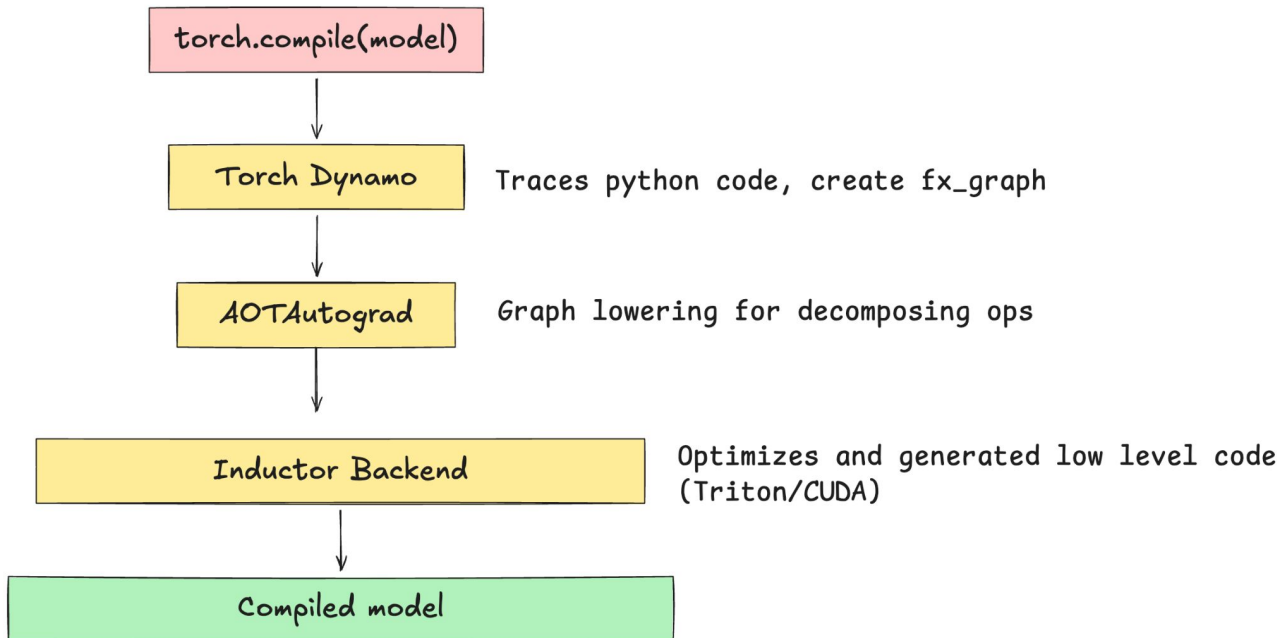
Control flow of vLLM



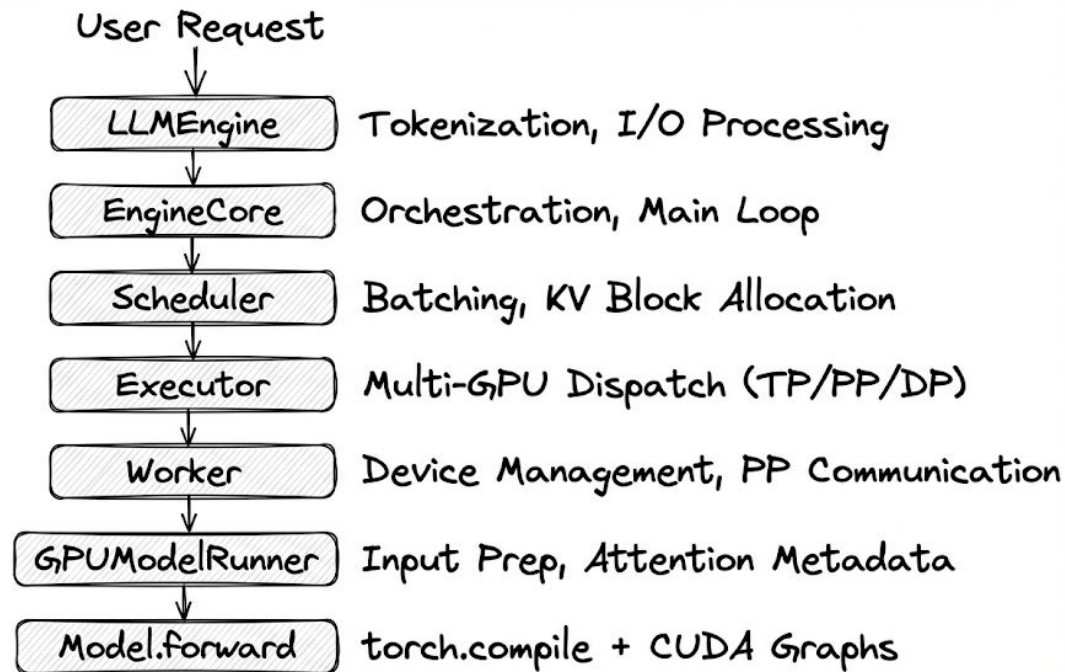
What is **torch.compile**?

torch.compile is a Just-In-Time (JIT) compiler that turns Python **PyTorch code** into highly optimized **kernel code** with a **single line decorator**

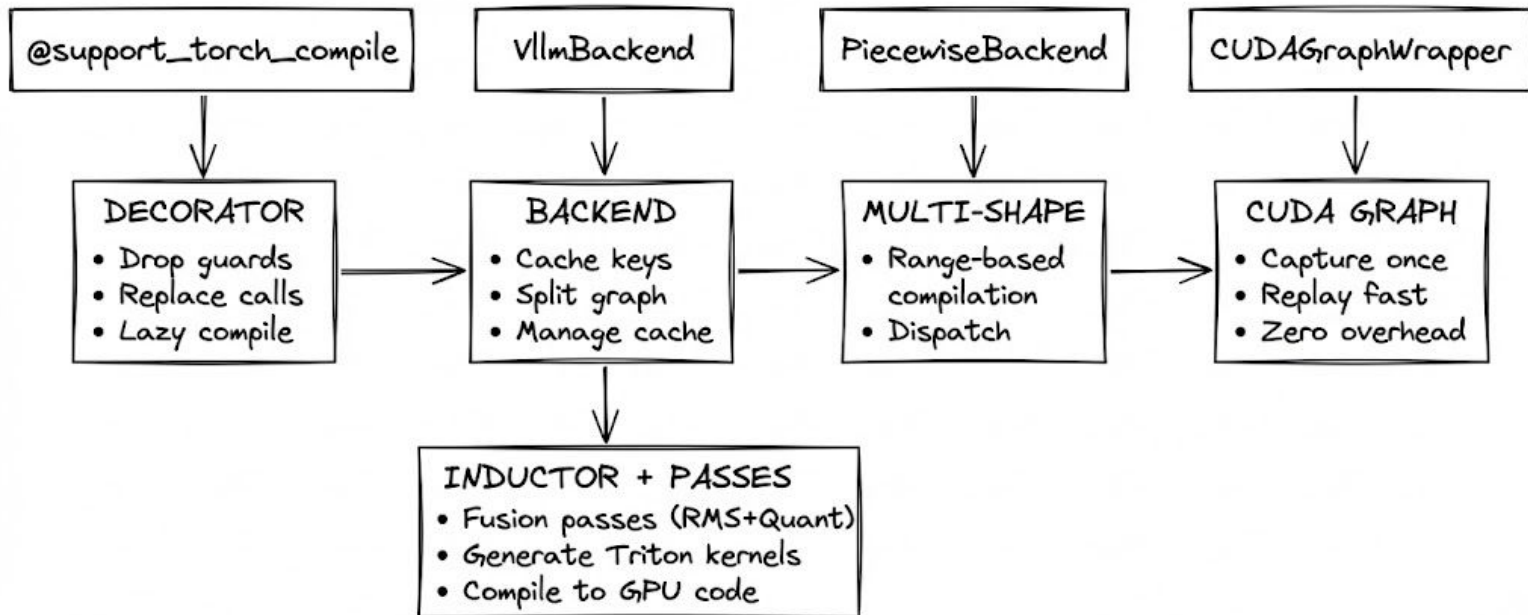
Control flow of **torch.compile**



where torch.compile meets vLLM



VLLM COMPILATION PIPELINE



LAYER 1: DECORATOR & WRAPPER
(decorators.py, wrapper.py)

INPUT: Model class with forward() method

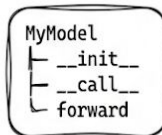
OUTPUT: Modified class ready for lazy compilation

```
@support_torch_compile(dynamic_arg_dins={"hidden_states": 0})  
class MyModel(nn.Module):  
    def forward(self, hidden_states, ...): ...
```

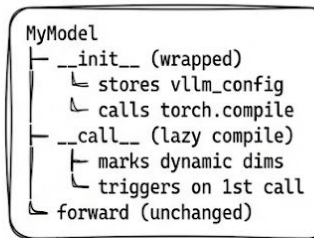


CLASS TRANSFORMATION

BEFORE:



AFTER:



KEY MECHANISM: Lazy Compilation

```
1st call: mark_dynamic() → torch.compile() → full pipeline  
2nd+ calls: direct dispatch (bytecode swap or compiled callable)
```

LAYER 2: VLLM BACKEND
(backends.py)

INPUT: FX Graph from Dynamo + example_inputs
OUTPUT: Optimized split graph with compiled submodules

VllmBackend.__call__()

STEP 1
Cache Key

env_hash
config_hash
code_hash
compiler_
hash
↓
cache_dir

STEP 2
Graph Split

FX Graph:
A B C D E
↓ split
S0 AT S1
compile|skip

STEP 3
Piecewise
Compile

For each
submodule:

Create
Piecewise
Backend
+
CUDA Graph
Wrapper

KEY MECHANISM: Graph Splitting

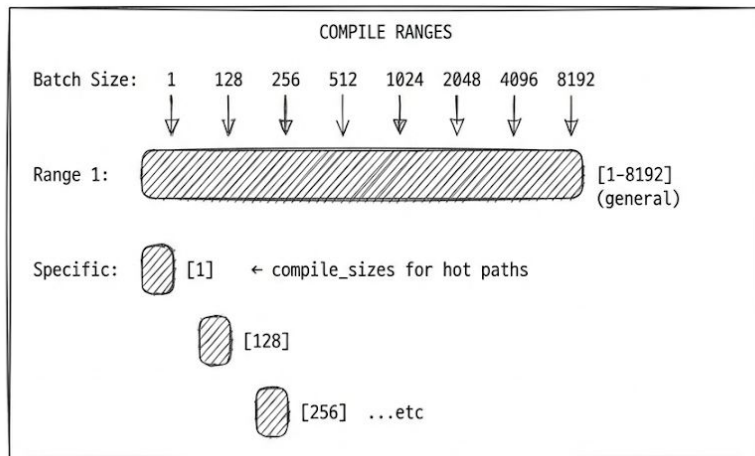
Split at attention ops → compile non-attention → skip attention
(Attention uses vLLM's custom PagedAttention, not torch.compile)

LAYER 3: PIECEWISE BACKEND
(piecewise_backend.py)

INPUT: Single subgraph + runtime batch size

OUTPUT: Correct compiled callable for that batch size

PROBLEM: Different batch sizes may need different optimizations



DISPATCH LOGIC:

```
__call__(runtime_shape=256):  
1. Check if 256 in compile_sizes → YES → use specific kernel  
2. If not compiled yet → trigger CompilerManager.compile()  
3. Return range_entries[256].runnable(*args)
```

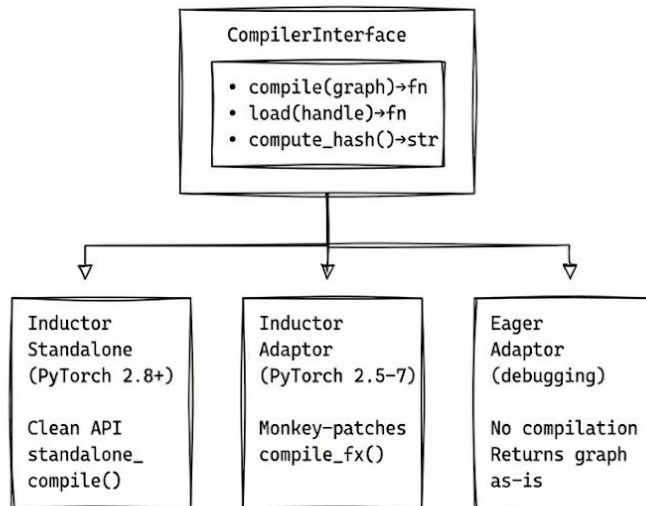
KEY MECHANISM: Shape-aware dispatch + lazy compilation

LAYER 4: COMPILER INTERFACE
(compiler_interface.py)

INPUT: FX Graph + example inputs + compile range

OUTPUT: Compiled callable (Triton kernels) + cache handle

COMPILER ABSTRACTION



COMPILATION FLOW (Inductor):

FX Graph → Lowering → Fusion Passes → Triton IR → PTX/CUBIN → Cache

LAYER 5: PASS MANAGER
(pass_manager.py)

INPUT: FX Graph (post-grad, pre-Triton)

OUTPUT: Optimized FX Graph with fused operations

FUSION EXAMPLE

BEFORE (3 kernels):

```
x = rms_norm(input)
y = fp8_quant(x)
z = gemm(y, w)
```



AFTER (1 kernel):

```
out = fused_rms_
      norm_quant
      (input)
z = gemm(out, w)
```

Memory: 3 reads + 3 writes

Memory: 1 read + 1 write

PASS EXECUTION ORDER:

1. NoOpElimination → Remove redundant reshape/slice
- ↓
2. RMSNorm+Quant → Fuse normalization + quantization
- ↓
3. Attention+Quant → Fuse attention output processing
- ↓
4. AllReduceFusion → Overlap communication + compute
- ↓
5. FixFunctional → De-functionalize (ALWAYS LAST)

LAYER 6: CUDA GRAPH WRAPPER
(cuda_graph.py)

INPUT: Compiled callable + batch descriptor
OUTPUT: GPU execution with minimal CPU overhead

PROBLEM: Even compiled code has Python dispatch overhead
SOLUTION: Record GPU commands once, replay without Python

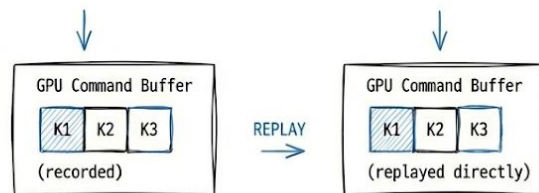
CAPTURE vs REPLAY

FIRST CALL (batch_size=256):

```
cuda_graph = CUDAGraph()
with torch.cuda.graph():
    output = runnable(*args)
# Records all GPU commands
entries[256] = (graph, out)
```

SUBSEQUENT CALLS:

```
if 256 in entries:
    entry.cuda_graph
    .replay()
# ~0 Python overhead
return entry.output
```



ENTRY CACHE:

```
concrete_cuda_graph_entries = {
    BatchDescriptor(256, ...): CUDAGraphEntry(graph, output),
    BatchDescriptor(512, ...): CUDAGraphEntry(graph, output),
    BatchDescriptor(1024, ...): CUDAGraphEntry(graph, output),
}
```

KEY BENEFIT: Eliminates kernel launch overhead (~5-20μs per kernel)

References:

- Anatomy of vLLM [Blog](#) (Must read!)
- vLLM [codebase](#) (Must read!)
- My previous [Talk](#) and [slides](#)
- vLLM paper [here](#)

THANK YOU

Reach out to me on:



@ayushsatyam146



@ayushsatyam146

